
ACSEFunctions Documentation

Release 0.1.0

Hazem Bakhshwain

Jun 08, 2026

CONTENTS:

1	acsefunctions package	2
1.1	Submodules	2
1.2	acsefunctions.functions module	2
1.3	Module contents	6
2	acsefunctions	7
3	Indices and tables	8
	Python Module Index	9
	Index	10

Welcome to the **ACSEFunctions** documentation.

ACSEFUNCTIONS PACKAGE

1.1 Submodules

1.2 `acsefunctions.functions` module

This module contains mathematical functions.

functions include such as factorial, exp, sinh, cosh, and tanh.

`acsefunctions.functions.bessel(alpha, x, N=100)`

Compute the Bessel function $J_{\alpha}(x)$ using the series expansion.

This function approximates the Bessel function using a truncated series expansion up to N terms.

Parameters

- **alpha** (*float*) – The order of the Bessel function (can be any real number).
- **x** (*float or array-like*) – The value(s) for which to compute the Bessel function.
- **N** (*int, optional*) – The number of terms in the series to use for the approximation (default is 100).

Returns

The computed value(s) of $J_{\alpha}(x)$.

Return type

`np.ndarray`

Notes

The Bessel function of the first kind is defined as: $J_{\alpha}(x) = \sum_{m=0}^{\infty} \frac{(-1)^m}{m! \Gamma(m + \alpha + 1)} \left(\frac{x}{2}\right)^{2m + \alpha}$ This implementation uses a truncated series expansion to approximate the result.

`acsefunctions.functions.cosh(x, N=10)`

Compute the hyperbolic cosine of x using the truncated Taylor series.

This function approximates the value of $\cosh(x)$ using a truncated Taylor series up to N terms. It supports scalar and array inputs, computing the result element-wise for arrays.

Parameters

- **x** (*float or array-like*) – The input value(s) for which to compute $\cosh(x)$. Can be a scalar or a NumPy array.
- **N** (*int, optional*) – The number of terms in the Taylor series to use for the approximation (default is 100). Higher values increase accuracy but also increase computation time.

Returns

The computed value(s) of $\cosh(x)$ as a NumPy array. If the input is scalar, a scalar value is returned; otherwise, an array of the same shape as the input.

Return type

np.ndarray

Examples

```
>>> cosh(0)
1.0
```

```
>>> cosh(1)
1.5430806348152437
```

```
>>> cosh(np.array([1, 2, 3]))
array([ 1.54308063,  3.76219569, 10.06766199])
```

Notes

- The calculation is performed using NumPy's vectorized

operations to support

both scalar and array inputs efficiently.

- The function incrementally computes each term in the series using the

previous

term, optimizing performance and reducing the risk of overflow.

acsefunctions.functions.**exp**(*x*, *N=100*)

Compute the exponential of *x* using the truncated Taylor series.

This function approximates the value of e^x using a truncated Taylor series up to *N* terms. It supports scalar and array inputs, computing the result element-wise for arrays.

Parameters

- **x** (*float or array-like*) – The input value(s) for which to compute e^x . Can be a scalar or a NumPy array.
- **N** (*int, optional*) – The number of terms in the Taylor series to use for the approximation (default is 100). Higher values increase accuracy but also increase computation time.

Returns

The computed value(s) of e^x as a NumPy array. If the input is scalar, a scalar value is returned; otherwise, an array of the same shape as the input.

Return type

np.ndarray

Examples

```
>>> exp(1)
2.718281828459045
```

```
>>> exp(np.array([1, 2, 3]))
array([ 2.71828183,  7.3890561 , 20.08553692])
```

Notes

- The calculation is performed using NumPy's vectorized operations to support both scalar and array inputs efficiently.
- The function incrementally computes each term in the series using the previous term, which helps minimize the risk of overflow and maximizes efficiency.

`acsefunctions.functions.fact(x)`

Calculate the factorial of a non-negative integer or NumPy array.

Parameters

x (*int or np.ndarray*) – The non-negative integer or array of integers for which the factorial is computed.

Returns

The factorial of x (x!) if x is a scalar, or an array of factorials if x is a NumPy array.

Return type

int or np.ndarray

Examples

```
>>> fact(5)
120
```

```
>>> fact(np.array([0, 1, 2, 3, 4]))
array([ 1,  1,  2,  6, 24])
```

Notes

Uses recursion and caching (@cache) for efficiency in scalar calculations. For NumPy arrays, the function is vectorized to handle each element.

`acsefunctions.functions.gamma(z)`

Compute the gamma function $\Gamma(z)$ using the Lanczos approximation.

This function uses the Lanczos approximation, which is known for its accuracy and efficiency in computing the gamma function.

Parameters

z (*float*) – The input value for which to compute the gamma function. Should be positive.

Returns

The computed value of $\Gamma(z)$.

Return type

float

Notes

The Lanczos approximation formula is:

$$\Gamma(z) \approx \sqrt{2\pi} * (z + g - 0.5)^{(z - 0.5)} * \exp(-(z + g - 0.5)) * A(z)$$

where $A(z)$ is a weighted sum of coefficients.

`acsefunctions.functions.sinh(x, N=10)`

Compute the hyperbolic sine of x using the truncated Taylor series.

This function approximates the value of $\sinh(x)$ using a truncated Taylor series up to N terms. It supports scalar and array inputs, computing the result element-wise for arrays.

Parameters

- **x** (*float or array-like*) – The input value(s) for which to compute $\sinh(x)$. Can be a scalar or a NumPy array.
- **N** (*int, optional*) – The number of terms in the Taylor series to use for the approximation (default is 100). Higher values increase accuracy but also increase computation time.

Returns

The computed value(s) of $\sinh(x)$ as a NumPy array. If the input is scalar, a scalar value is returned; otherwise, an array of the same shape as the input.

Return type

np.ndarray

Examples

```
>>> sinh(0)
0.0
```

```
>>> sinh(1)
1.1752011936438014
```

```
>>> sinh(np.array([1, 2, 3]))
array([ 1.17520119,  3.62686041, 10.01787493])
```

Notes

- The calculation is performed using NumPy's vectorized operations to

support

both scalar and array inputs efficiently.

- The function incrementally computes each term in the series using the

previous

power of x , which helps minimize computational overhead.

`acsefunctions.functions.tanh(x, N=10)`

Compute the hyperbolic tangent of x using $\sinh(x)$ and $\cosh(x)$.

This function calculates $\tanh(x)$ as the ratio of $\sinh(x)$ and $\cosh(x)$, using a truncated Taylor series up to N terms. It supports scalar and array inputs, computing the result element-wise for arrays.

Parameters

- **x** (*float or array-like*) – The input value(s) for which to compute $\tanh(x)$. Can be a scalar or a NumPy array.
- **N** (*int, optional*) – The number of terms in the Taylor series used for $\sinh(x)$ and $\cosh(x)$ approximations (default is 100). Higher values increase accuracy but also increase computation time.

Returns

The computed value(s) of $\tanh(x)$ as a NumPy array. If the input is scalar, a scalar value is returned; otherwise, an array of the same shape as the input.

Return type

np.ndarray

Examples

```
>>> tanh(0)
0.0
```

```
>>> tanh(1)
0.7615941559557649
```

```
>>> tanh(np.array([1, 2, 3]))
array([0.76159416, 0.96402758, 0.99505475])
```

Notes

- This function uses the $\sinh(x)$ and $\cosh(x)$ functions previously defined, which are based on truncated Taylor series expansions.
- The calculation is performed using NumPy's vectorized operations to

support

both scalar and array inputs efficiently.

1.3 Module contents

The `acsefunctions` package provides mathematical functions.

These functions include factorial, exponential, and hyperbolic functions.

ACSEFUNCTIONS

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

a

`acsefunctions`, [6](#)

`acsefunctions.functions`, [2](#)

A

`acsefunctions`
 module, [6](#)
`acsefunctions.functions`
 module, [2](#)

B

`bessel()` (*in module `acsefunctions.functions`*), [2](#)

C

`cosh()` (*in module `acsefunctions.functions`*), [2](#)

E

`exp()` (*in module `acsefunctions.functions`*), [3](#)

F

`fact()` (*in module `acsefunctions.functions`*), [4](#)

G

`gamma()` (*in module `acsefunctions.functions`*), [4](#)

M

module
 `acsefunctions`, [6](#)
 `acsefunctions.functions`, [2](#)

S

`sinh()` (*in module `acsefunctions.functions`*), [4](#)

T

`tanh()` (*in module `acsefunctions.functions`*), [5](#)